

Q1. What is type checking? Demonstrate static vs dynamic typing using Java.

Type checking:

Type checking is the process by which a compiler or interpreter verifies that the operations performed in a program are consistent with the types of the values involved (e.g., ensuring a string is not added to a boolean where that is not permitted). It prevents type errors and increases program reliability.

Static typing:

In static typing, the type of every variable is checked at compile time, before the program runs. Java is a statically typed language:

```
int x = 10;
x = "hello"; // Compile-time error:
              // incompatible types: String cannot be converted to int
```

This error is caught by the compiler before the program ever executes.

Dynamic typing (simulated in Java):

In dynamically typed languages (e.g., Python, JavaScript), type checks happen at run time, and a variable's type can change during execution. Java itself is not dynamically typed, but this behavior can be illustrated using the generic Object type together with runtime casting:

```
Object obj = 10;    // obj holds an Integer at runtime
obj = "hello";      // now obj holds a String at runtime
                    // allowed because obj is declared as Object

if (obj instanceof String) {
    String s = (String) obj; // type resolved/checked at run time
}
```

Here, the actual type held by obj is only determined and checked while the program is running, which mirrors how dynamically typed languages behave.

Q2. Write a Java program showing variable scope.

```
public class ScopeDemo {
    static int classVar = 100;          // class (static) scope

    public static void main(String[] args) {
        int methodVar = 10;            // method (local) scope
    }
}
```

```

System.out.println("classVar: " + classVar);
System.out.println("methodVar: " + methodVar);

{
    int blockVar = 5;           // block scope
    System.out.println("blockVar: " + blockVar);
    System.out.println("methodVar visible here: " + methodVar);
}
// blockVar is NOT accessible here; it went out of scope

for (int i = 0; i < 3; i++) {    // i has loop scope
    System.out.println("i = " + i);
}
// i is NOT accessible here
}
}

```

This program illustrates four levels of variable scope in Java:

- Class/static scope: classVar is accessible throughout the class and to any method within it.
- Method (local) scope: methodVar is accessible anywhere within the main method, including nested blocks inside it.
- Block scope: blockVar exists only within the enclosing { } block and is discarded once that block ends.
- Loop scope: the loop variable i exists only for the duration of the for loop.

Q3. Explain the difference between binding time and run-time with code examples.

Binding time:

Binding time refers to the point during a program's life cycle at which a property — such as a variable's type, a value, or a memory address — becomes fixed ("bound"). Common binding times include language-design time, compile time, link time, and run time.

Compile-time binding example:

```

final int MAX = 100; // value bound to MAX at compile time
                  // (a compile-time constant)

```

Run-time binding example:

```

import java.util.Scanner;

public class BindingDemo {
    public static void main(String[] args) {

```

```
Scanner sc = new Scanner(System.in);
int age = sc.nextInt();    // value bound to age at RUN TIME,
                           // depending on user input
System.out.println("Age entered: " + age);
}
}
```

In the first example, the value of MAX is known and fixed as soon as the program is compiled — this is compile-time binding. In the second example, the value of age cannot be known until the program is actually running and the user provides input — this is run-time binding.

More generally: binding time is the concept describing when a binding occurs, while run time is one specific, and the latest possible, point at which binding can happen (as opposed to earlier points like compile time or link time). Run-time binding offers more flexibility (e.g., dynamic method dispatch, polymorphism) but can be slightly less efficient than binding done earlier.